

Trainer Apis

In application Trainer is a user and trainer is associated with multiple studios and also can be registered as its own, So for a Trainer user id there will be multiple association with trainer id. Trainer id is used in all api calling where and required, trainer ids are used for internal purpose.

Trainer details are stored in multiple tables

- Users table has trainer basic information which is used for login.
- Studio_trainers contains trainer and studio association details, one to many relation to user and Studio_trainers as per how many studios he is associated with.
- Studio_trainers_info table has trainer related information like certification, trainer bio information One to One relation between user and Studio_trainers_info table user id and Studio_trainers_info id are same
- Studio_trainer_slots has information about time slots chosen by this trainer w.r.t studio and his own also.

PUT

[/api/trainer/{userId}/update](#)

To update Trainer information, like bio, certification, image etc. if certificate or image not provided as part of request, will remain as provided earlier or existing.

- As part of update suppose to choose freely available slot and slot already chosen w.r.t to studio or self registered.
- If no change in slot modification, as part of request it is mandatory to send an already chosen slot.

Addition information to use above api efficiently from GUI

API	Details
/api/trainer/{userId}	Get basic trainer information
/api/trainer/{userId}/certificate	Get Certification if want to show certification as part of update screen
/api/trainer/{userId}/studio	• Get List of studio associated by this trainer • If trainer also registered self, then one entry come with null in the response list • If null entry exist, by default studio

	drop down should not select any thing • should only show this list as part of update • should not allow new studio to map
/api/trainer/{userId}/time-slots	• Get time slots associated with the studio if studio id given as part of request, else will respond with self registered slots if trainer self registered
/api/trainer/{userId}/time-slots/free	Will list freely available slots

GET

</api/trainer/{userId}>

Get Basic Trainer information like name, trainer information, has a certificate uploaded or not.

It will not include associated studios information and selected time slots

GET

</api/trainer/{userId}/certificate>

To get certificate in base64 encode string

PATCH

</api/trainer/{userId}/certificate>

To Upload new certificate, if already a certificate uploaded will be override with new one.

GET

</api/trainer/{userId}/classes>

- List scheduled classes for a trainer
- Optional query string time-slot
 - If given, it list Students name, studio name only for given time slot
 - If not given, it list all Students name, studio name for the given trainer user id

GET

</api/trainer/{userId}/classes/count>

Get total counts of classes secluded for the given trainer user id, should **not** use this for pagination.

GET

</api/trainer/{userId}/studio>

Get Associated Studio id and name for given trainer user id. If the trainer registered himself also there is one null entry that will come, meaning the user self registered.

GET

</api/trainer/{userId}/time-slots>

Get Trainer Available Time Slots which can use to filter or map users with trainer

Optional query parameter `studio-id`

- If given it will response time slots choosan by trainer for this studio
- If not provided, will response time slots choosan by trainer as part of self register not associated with any studio.

GET

</api/trainer/{userId}/time-slots/all>

Get All choosan time slots by the trainer across all, with studios and self.

For example trainer registered with studio 1 and 2 and self register with time slots 6,7 for studio 1 and 8 and 9 for studio 2 and 18 for self.

Then this API will give all above 6,7,8,9 and 18

And the above api will give 18 if query parameter not provided

6,7 will give if query parameter is 1

8 and 9 for query parameter 2

GET

</api/trainer/{userId}/time-slots/free>

Will list all time slots from master table minus slots already chosen by the trainer with self and with other studios.

Free slots = master time slots - time slots already chosen by the trainer.

For example master has time slot 5,6,7,8,9,18 and 19, the with continuation above example trainer this will give 5 and 19

GET

</api/trainer/{userId}/user-count>

Get User count for given studio trainer's user id

Optional query parameter `studio-id`

- If given it will response count for this trainer associated with given studio id
- If not provided, will give all user count associated with this trainer across all studios and assigned self.
-

POST

</api/trainer/add>

Allow to add a new trainer, can use this for self register or admin can add trainer to system.

GET

</api/trainer>

Get basic Trainers information with studio name, for trainer self registered then studio details will be empty.

This support pagination optional parameter `page` and `size`

Optional query parameter `studioId`, if provided will response only trainer related to studio, otherwise will list all trainers

GET

</api/trainer/count>

Get Trainers count

Optional query parameter `studioId`, if provided will response only trainer count related to studio, otherwise will list all trainers count

This api can use to find [no.of](#) pages to display above api response with pagination

For example if count is 75 and per page expecting to display 10 rows then $75/10 = 7.5$, so it will 8 pages

GET

</api/trainer/list>

Get Active Trainers user id and name with studio id and name, for self registered trainer studio id and name will be null.

For example a trainer registered as self and also mapped to a studio, then in the list it will be 2 entries one with studio id and name and another studio id and name as null

PUT

</api/trainer/map>

Used to map a registered trainer with another studio.

Input will be

- studio id
- trainer user id
- student limit to this trainer under this studio. (limit should not exceed overall count allocated to studio)
- Subscribed start and end date.
- Time slot chosen by this trainer for this studio (use </api/trainer/{userId}/time-slots/free> to get free slots)

GET

</api/trainer/overview>

Gives information to show in trainers overview in dashboard

As response will get

- List of trainers with total user count associated with trainer
- Number of trainer pending for approval
- Total online students count

GET

</api/trainer/status/{status}>

Get All Trainers basic information with studio name for given Status

This support pagination optional parameter `page` and `size`

GET

</api/trainer/status/{status}/count>

Get Trainers count for given status,

This api can use to find [no.of](#) pages to display above api response with pagination

For example if count is 75 and per page expecting to display 10 rows then $75/10 = 7.5$, so it will 8 pages

PATCH

</api/trainer/update-status>

Update Trainer Status, can use to change status of trainer like Approval, Rejected

PATCH

</api/trainer/update-subscription>

Update Trainer Subscription

- For self registered trainer only trainer user id to send, no need to send studio id
- Studio id to be sent, if subscription is updating for trainer with studio.

User Asana Plan Apis

APi to manager asana plans fo a user

POST

</api/users/create-asana-plan>

This can use to save manually hand crafted asana plan for user by trainer.

If any existing active plan will be marked to deleted, before inserting new plan

GET

</api/users/get-asana-plan/{userId}>

Will return active asana plan information with title, description, number of asanas, duration and each asana with asana code, short description and image in encoded string format if exist otherwise "Image not found" string will come

PUT

</api/users/update-asana-plan>

Use this API to update existing manually crafted or recommendation engines to generate asana plans for a user.

If any existing active plan will be marked to deleted, before inserting new plan

GET

</api/users/generate-asana-plan/{userId}>

This will invoke a recommendation engine based on the user profile and generate an asana plan and save it. On success full generate it will response message with asana plan generated success fully, other wise will give appropriate information.

If any existing active plan will be marked to deleted, before inserting new plan

After successful response, call [get-asana-plan](#) to view the generated asana plan.

Asana Master Apis

Following APIs are used for managing Asana Master Table

GET

</api/v1/asana>

List all entries from asana master with asana id, name, code, description, detail and benefit and contain only image name, it will give image content.

POST

</api/v1/asana>

Add a new asana information to master with all required details code, name, details, shortDescription, category, type, benefits, minRepetitions, maxRepetitions, frequency, video (video path), image (image path)

GET

</api/v1/asana/{asanaCode}>

Get asana details for given asana code, including image in base64 encoding string formate, no video details

PUT

</api/v1/asana/{asanaCode}>

Used to update asana details for given asana code, with given asana details, it will not update asanaCode, even if it is provided as part of request.

DELETE

</api/v1/asana/{asanaCode}>

Delete a Asana details for given asana code

POST

</api/v1/asana/by-asanaCodes>

Get a list of Asana details for a given list of asana codes, its same as GET call of </api/v1/asana> but it has image information in base64 format only for given asana code.

POSTs

</api/v1/asana/upload-excel>

Use this api to upload full asana master data by using excel sheet,

API will truncate existing data in the master table and save this information.

Excel sheet should follow following template

Asana Code	Min Repetition	Max Repetition	Repetition Freequency	Asana Benefits	Asana Image	Asana Details	Short description
------------	----------------	----------------	-----------------------	----------------	-------------	---------------	-------------------

Authentication Apis

This APIs can use to perform Authentication related actions

POST

</api/auth/change-password>

Api to change password, this should only be used when a user successfully login and wants to change password.

Should provide user login id (phone number), old password and new password as part of request

POST

</api/auth/login>

Api for login

POST

</api/auth/logout>

Api for log out

POST

</api/auth/refresh>

Api to refresh JWT token for already logged in user.

POST

</api/auth/request-reset-mail>

API to send reset password token mail to user for provider user id.

POST

</api/auth/reset-password>

This is to reset password with received rest token in email

this should only be used when a user try to reset with out login.

Should provide user login id (phone number), old password (token received in mail) and new password as part of request

Common Apis

Following APIs are used to list static data, normally used for drop down in many places like, trainer registration, user registration etc.

GET

</api/common/age-groups>

Gives list of values from group name **Age Group** in master table.

GET

</api/common/gender-list>

Gives list of values from group name **Gender** in master table.

GET

</api/common/hereditaries>

Gives list of values from group name **Hereditary** in master table.

GET

</api/common/meal-types>

Gives list of values from group name **Meal Type** in master table.

GET

</api/common/roles>

Get All User Roles, it come from Enum value defined in code

GET

</api/common/sleep-patterns>

Gives list of values for group name **Sleep Pattern** in master table. Get Sleep Patterns

GET

</api/common/status-list>

Get All User Status like PENDING, APPROVED etc, it come from Enum value defined in code

GET

</api/common/stay-types>

Gives list of values from group name **Stray Type** in master table.

GET

</api/common/stress-level>

Gives list of values from group name **Stress Level** in master table.

GET

</api/common/surgeries>

Gives list Surgeries and Injuries for group name **Surgeries** in master table.

GET

</api/common/symptoms>

Gives list health conditions form group name **Symptoms** in master table.

GET

</api/common/user-levels>

Gives list Experience Levels form group name **Experience Level** in master table.

GET

</api/common/yoga-goals>

Gives list Yoga Goals form group name **Yoga Goals** in master table.

Enum Values Apis

Generic API to add new group to master enum value and get with group name if common api not available for such group

POST

</api/enum>

Create new group with respected values, will be stored in master table, recommended first delete group from master by using delete api and use this api to save all again for that group

For example if Group Symptoms add new values, delete Symptoms first and add Symptoms with old and new values again.

GET

</api/enum/{group}>

Get All Values for a given group

DELETE

</api/enum/{group}>

Delete All Values for a given Group

Master Time Slot Apis

APIs to manager Master time slots

Master time slots are used to pick time slot for user, for **trainer time slots please check appropriate apis in Trainer APis**

GET

</api/common/time-slot>

Get all time slots from master, normally used to list down in user registration form.

POST

</api/common/time-slot>

Add a new time slot for the master table.

DELETE

</api/common/time-slot/{slot}>

Remove time slot from master table